From Bounded Rank-N Plumbing to Typed, Feedback-Directed Synthesis

Future directions for Djex and Leant

A review of their coordinated git history through 10 August 2026

Repositories reviewed

VladimirReshetnikov/Djex
VladimirReshetnikov/Leant

Review snapshot

Djex main: `f50b5eac86726c7530d95e48c27101b7c0fa25c9`
Leant main: `0160237e4bdf6e342a55f5a86ce5d5292728024f`
Leant submodule `lib/Djex`: the same Djex head

Technical report prepared 10 August 2026

Status, 14 August 2026: several parts of the proposed target architecture have since been implemented in Djex and consumed by Leant: typed candidate nodes with a sealed term graph and stable node, occurrence, and certificate identities; the bounded source-certificate table with atomic graph association; carrier-aware canonical fingerprints; and a first checked behavioral domain (finite list-spine lengths with live Z3 ranking, bounded input-box validation, and a binary-product extension). Adaptive occurrence planning, the shared subgoal DAG, recursor skeletons, and the staged dependent fragment remain open. The dated reports under `docs/reports/` in both repositories record the exact boundaries.

Status, 19 August 2026: the behavioral domain has since also gained opt-in hard filtering (`:synth --behavior-mode filter`, in which only an independently replayed counterexample rejects a candidate), command-local counterexample banks, progressive same-run filter batching, and sealed descriptor-bound Z3 launches with effective-ID and execve-check access variants. The planning-layer items above remain open.

Abstract

The recent histories of Djex and Leant show an unusually productive co-development loop. Leant exposes a concrete Lean-side synthesis failure; that failure identifies information erased at the language-neutral boundary; Djex then generalizes the missing invariant or search rule; and Leant advances its pinned submodule, preserves the source semantics that remain Lean-specific, and closes the loop with kernel-checked transcripts. In roughly the last two days alone, this loop widened bounded instantiation to six leading binders, carried higher-kinded class arguments, preserved contextual rank-N provider assignments, enforced assignment fidelity through nested and sibling datatype fields, specialized provider results from exact term evidence, and finally retained the structural identities of `Prod`/`Sum` through higher-kinded substitutions.

The success of that loop also reveals its next scaling limit. More and more semantic information is transported beside an intentionally erased Haskell-like search term, then reconstructed in Leant's renderer by fitting generated syntax back against the original Lean fragment. This works, and Lean verification makes it safe, but it creates a growing *semantic recovery tax*: provider metadata must be coupled to individual occurrences; forall visibility and sort domains need parallel side channels; result types are rediscovered from term arguments; and canonical family identity needs special handling at every stage.

The central recommendation of this report is therefore not simply to raise the six-binder bound, add a sextic occurrence frontier, or teach another structural constructor to the current adapters. The next architectural layer should be a frontend-neutral, typed candidate graph carrying stable occurrence identities, expected types, source-provided certificates, explicit residual obligations, and a structured account of projection loss. Djex can continue to perform bounded structural search over a compact logical projection; Leant can keep Lean expressions, universes, binder visibility, type-class evidence, and indices opaque where appropriate; and the two can communicate through typed skeletons instead of post-hoc textual repair.

On that foundation, the most promising functional extensions are: an adaptive rather than fixed-degree rank-N plan search; a shared hash-consed subgoal and candidate DAG; generic correlated-substitution certificates; a frontend-declared algebra of structural families; first-class evidence and equality obligations; recursor-skeleton synthesis for recursive data; a staged dependent-index fragment; semantic provider retrieval at Mathlib scale; and genuinely cooperative scheduling between Djinn and Exference. The report gives concrete data types, algorithms, invariants, migration steps, tests, and a priority-ordered roadmap designed to preserve the projects' current fail-closed and independently verified posture.

Contents

1 Scope, method, and terminology

This report reviews the current source trees, the public architecture and proposal documents, and the recent commit histories of `VladimirReshetnikov/Djex` and `VladimirReshetnikov/Leant`. The emphasis is not on listing every commit. It is on finding repeated causal patterns across the two repositories: what class of Lean goal or provider exposed a gap; which repository supplied the general fix; what information had to cross the boundary; which conservative guard was added; and what that sequence implies for the next abstraction.

The review snapshot is the pair of main-branch heads shown on the title page. At that point, Leant pins Djex exactly at `f50b5eac86726c7530d95e48c27101b7c0fa25c9`. This is important: the two histories are not merely conceptually related. The Leant history records a series of exact gitlink advances to freshly reviewed Djex capabilities, followed by Lean 4.31 transcripts that exercise those capabilities through the complete serializer–engine–renderer–elaborator path.

Throughout the report:

- *search type* means the type representation consumed by one of the synthesis engines after frontend translation;
- *source type* means the richer Haskell or Lean type known before projection;
- *candidate* means a generated term or term skeleton together with whatever evidence and obligations are needed to interpret it;
- *positive soundness* means that every displayed term is accepted by the source language's checker (GHC in Djex integration tests, the Lean elaborator and kernel in Leant);
- *negative completeness* means that an exhausted search justifies a non-inhabitation claim for a stated fragment, rather than merely failing to find a term under finite bounds;
- *projection loss* means source structure deliberately hidden from the engine: an opaque dependent atom, an unopened quantifier occurrence, an omitted eliminator, a truncated provider inventory, or similar information.

The report deliberately distinguishes *source-language semantics* from *search semantics*. The goal is not to turn Djex into a Lean elaborator, nor to make Leant duplicate both synthesis engines. The goal is to choose a stronger interface at which each side can retain what it uniquely knows.

2 Current architecture at the reviewed heads

2.1 Djex: two engines behind a checked neutral foundation

Djex combines Djinn, a complete terminating LJT-style search for its admitted intuitionistic fragment, and Exference, a ranked heuristic search with explicit resource controls and type-class evidence resolution. The merger is no longer just a shared executable. The package exposes a parser-independent synthesis foundation containing validated names, kinds, types, constraints, declarations, environments, generated expressions, queries, diagnostics, and search batches.

Several current public types are especially relevant to future work. The shared source type is structurally richer than the original engines:

Listing 1: Current shared source-type shape (abridged).

```
data Type variable
  = TypeVariable variable
  | TypeConstructor Name
  | TypeApplication (Type variable) (Type variable)
```

```
  | FunctionType (Type variable) (Type variable)
  | TupleType Boxity [Type variable]
  | ForallType
      [variable]
      [Constraint (Type variable)]
      (Type variable)
```

It distinguishes flexible inference variables from rigid skolems; provides capture-avoiding substitution and binder normalization; preserves explicit applications; and gives saturated functions and tuples canonical structural forms while still exposing a constructor-application view for higher-kinded unification.

The shared generated-term syntax has also grown beyond a first-order lambda calculus. It contains patterns, lambdas, applications, visible type applications, tuples, holes, lets, and cases. A complete application spine can interleave term and visible-type arguments in source order. Specified visible type arguments are closed and alpha-normalized rather than being raw strings.

Most importantly for extensibility, a public candidate already has a natural place for information beyond the rendered term:

Listing 2: The existing candidate envelope is an opening for a richer protocol.

```
data Candidate ty details output = Candidate
  { candidateOutput              :: output
  , candidateResidualConstraints :: [Constraint ty]
  , candidateDetails             :: details
  }
```

The current `details` parameter is backend-owned and the residual constraints are type-class-shaped. The proposed architecture in this report extends, rather than discards, that idea: make residual obligations more general and give candidate details a stable neutral core that frontends and both engines can share.

The provider-instantiation boundary is another major asset. It has evolved from independent scalar candidates into exact, ordered, correlated vectors, and then into kinded vectors:

- `ProviderInstantiationCandidate` contributes one provider-local type to a pool;
- `ProviderInstantiationAssignment` preserves a complete leading binder assignment in source order;
- `KindedProviderInstantiationAssignment` pairs each selected type with its exact ground kind.

This is already a small certificate language: an external frontend asserts a provider identity and a correlated substitution; the checked backend validates session identity, arity, kinds, closure, and structural fidelity before using it. Section 6.5 argues that this should become the template for all source-provided semantic evidence.

2.2 Leant: semantic serializer, search adapter, and independent checker

Leant is a Lean REPL whose `:synth` pipeline links Djex in-process. The architecture intentionally keeps the engine boundary narrow: `Leant.Synth.Engine` is the only synthesis module importing Djex, while `Fragment`, `Render`, and the backend worker own Lean-specific translation and verification.

The input fragment is substantially richer than a simple propositional encoding. It distinguishes:

- arrows, products, sums, top, and bottom;
- explicit or implicit sort quantifiers and erased instance binders;
- exact contextual class applications with ground-kinded arguments;
- bound and nominal proper-type applications;
- parameterized finite inductive families;
- parameterized recursive families with bounded constructor metadata;
- safe and unsafe opaque atoms; and
- a depth sentinel that makes truncation explicit.

The serializer runs inside Lean, after elaboration, and therefore has access to information no Haskell parser or string renderer can reliably reconstruct: exact constants, binder visibility, whether a domain is `Prop`, `Type`, or a general `Sort`, active instance heads, constructor inventories, recursive occurrences, and definitional reduction through the current environment.

The output path is just as important. Leant does not trust an engine term. It fits the generated expression against the original fragment, restores exact Lean names and binder positions, enumerates bounded source spellings where semantic information is genuinely ambiguous, and sends each surviving term back to Lean for elaboration against the exact goal. A candidate is displayed only after that check succeeds.

This independent checker is what permits aggressive positive search without compromising correctness. It does *not*, however, make arbitrary semantic erasure free. Every erased fact that is later necessary for an otherwise valid term becomes either a dropped candidate or another renderer-side reconstruction rule. The size and recent history of `Leant.Synth.Render` and `Leant.Synth.Fragment` are evidence that the cost has become architectural rather than incidental.

2.3 Current scheduling and honesty rules

At the reviewed head, the main interactive policy includes:

- a provider-free structural baseline;
- live-provider fallback, with bounded discovery and staged widths;
- Djinn, Exference, or a combined portfolio;
- at most five displayed terms, twelve fresh candidate groups tried per standalone engine, and a combined frontier preserving twelve opportunities for each engine;
- an internal candidate collection window of sixty;
- strict Exference search first, followed by an allow-unused retry only after a strict miss; and
- Lean verification of every textual variant before display.

Negative verdicts follow a stricter rule than positive ones. Exference never provides a non-inhabitation proof. Djinn exhaustion is exposed as a sound refutation only when the logical projection is complete and the Lean translation did not hide concrete structure in an unsafe atom. Provider lanes may still find a constructive term after a provider-free refutation; if they do not, Leant restores the retained refutation before considering the classical fallback.

This separation between positive verification and negative completeness is one of the strongest design decisions in both projects. Future extensions should make it more expressive, not weaken it.

3 The co-evolution pattern in recent history

3.1 A condensed cross-repository timeline

Table 1 groups the most informative recent changes into capability loops. Dates are commit dates; the short hashes link to the exact changes.

Table 1: Representative capability loops in the coordinated history.

Date	Theme	Djex movement	Leant movement
2026-08-06	Bounded rank-N occurrence plans	`d728719f` adds quintuple-open and quintuple-opaque tails, capped at 512 views per orientation and exhaustive through eleven independent sites.	`80f123a4` and `3fcf7cfa` advance the submodule and verify genuinely non-prefix ten- and eleven-site Lean examples.
2026-08-09	Query-closed and correlated instantiation	`227bdc52` adds closed query-subtree choices for local schemes; `077f767f` through `844e548d` add result-correlated guarded impredicative choices while preserving historical prefixes.	`21c0bab6` pins the reviewed engine; `e09bded3` and `15fb0962` repair explicit provider heads and nested fitting; `c333f2c0` merges live Lean coverage.
2026-08-10	Scalar global aggregate elimination	`db25170b` lets Exference derive boxed-tuple eliminators from all checked type sources and deconstruct a scalar global once.	`690277ca` carries exact provider result fragments through case and let; `9893fef5` normalizes the engine's intrinsic pair patterns; `65b93bcc` merges the checked path.
2026-08-10	Provider-result specialization	`e10f7f01` and `d918334a` provide one strict, lazy, mixed term/type application-spine constructor.	`892117dc` through `9189d509` infer provider-result substitutions from exact term evidence, thread them through full higher-order applications, and preserve trailing quantified structure.
2026-08-10	Exact contextual assignments and fidelity	`03aa3820` accepts closed contextual polytypes in exact vectors but marks selected types through nested structures; `103fdd39` requires every marker-bearing sibling field to retain the selected identity.	`2979dbe2` introduces an exact-assignment-only nominal Lean class context and private methodless engine classes; `647f3940` pins the strengthened fidelity boundary.

Date	Theme	Djex movement	Leant movement
2026-08-10	Occurrence-local rendering metadata	The canonical visible type is intentionally language-neutral; Djex keeps specified visible arguments alpha-normalized and bounded.	`888e2b40` retains forall visibility and sort-domain tags; `fe4b2932` couples one metadata choice to each provider occurrence so repeated uses can select independently.
2026-08-10	Higher-kinded contextual evidence	`c5afc68a` preserves explicit class-parameter kinds in checked shared environments; `0ea49f9b` checks a rank-N assignment constrained by a type constructor.	`1668c956` serializes residual kind arity, reconstructs private kinded classes, plans a nominal head at total arity, and rejects conflicts source-locally; `d3662c31` merges Lean 4.31 verification.
2026-08-10	Six-binder common boundary	`66955da7` raises the single public provider/instantiation width from five to six so Djinn and Exference advance together; seven remains a productive checked rejection.	`d04c1a8f` keeps adjacent Lean forall nodes as one six-binder scheme and derives producer, parser, engine, and renderer bounds from Djex; `de4f562c` merges all-engine and live tests.
2026-08-10	Canonical structural higher-kinded identity	`5a550eb4` retains the bare boxed pair constructor through Djinn exact substitutions; `f50b5eac` merges both-engine and GHC-checked support.	`0dbe08a7` separates canonical nominal context evidence from legacy fragments and maps exact `Prod`/`Sum` heads to their structural engine identities; `0160237e` merges the Lean-verified result.

3.2 The recurring five-step capability loop

The timeline shows a stable development method:

1. **A precise source-language counterexample appears.** The useful failures are rarely abstract. They are types such as a provider whose visible type argument is itself quantified, a provider returning a tuple containing a rank-N field, a class argument whose kind is `Type -> Type`, or a canonical `Prod` assignment consumed later in saturated form.

2. **Leant preserves just enough semantic evidence to state the failure faithfully.** Exact names, binder visibility, sort domains, instance-head correlations, source fragments, and constructor inventories are kept rather than guessed from pretty text.

3. **Djex generalizes the missing search or boundary invariant.** Examples include exact ordered assignment vectors, shared kinded classes, a strict mixed application spine, bounded closed/correlated instantiation, scalar aggregate elimination, and structural assignment-fidelity checks.

4. **Leant advances the submodule and adapts only the remaining Lean-specific semantics.** The renderer restores source names and syntax; the serializer supplies metadata that cannot be language-neutral; the engine module remains the narrow adapter.

5. **The exact source-language case is independently checked.** Djex uses public-facade and

GHC compilation regressions. Leant uses pure boundary tests and Lean 4.31 transcripts. Malformed, over-wide, phantom, dictionary-dependent, and unsupported controls are generally added beside the positive case.

Finding 1: the synergy is not accidental coupling; it is a productive division of semantic authority.

Djex is best placed to own finite search, alpha-safe neutral syntax, correlated substitutions, common bounds, and engine-independent invariants. Leant is best placed to own elaborated Lean identities, universes, binder roles, instance provenance, constructor/recursor metadata, and final kernel checking. The future architecture should formalize this division rather than trying to collapse it.

3.3 What the same history warns about

The loop is productive, but the shape of the fixes has changed. Early work added logical rules or widened a clearly bounded family. The most recent work is dominated by keeping semantic side channels synchronized across:

- provider discovery and exact assignment parsing;
- query-wide family planning;
- the Djinn and Exference projections;
- generated visible type applications;
- occurrence-local fitting;
- Lean binder-domain rendering; and
- final elaboration.

The `Prod/Sum` higher-kinded change is illustrative. A saturated product is structural in both engines; a bare `Prod` is a higher-kinded source constant; an exact assignment may carry the bare or partially applied head; a contextual class may mention it; provider planning must not close it as an ordinary proper-type atom; the engine must map it back to the same structural identity used by saturated products; and the renderer must retain the exact Lean source choice. The implementation is correct and conservative, but each new family with analogous dual structural/nominal behavior would repeat much of that path.

Likewise, provider-result specialization now uses exact term arguments to infer fragment substitutions, re-applies them before inspecting each residual constructor, fits later arguments against newly specialized domains, and recurses through nested provider applications. This is valuable functionality. Architecturally, however, it is type propagation over an untyped generated expression—that is, a small bidirectional elaborator implemented after search because the candidate edge did not carry the information earlier.

Finding 2: the dominant scaling problem is now semantic erasure followed by source-specific recovery.

The present boundary is no longer too weak because it lacks another expression constructor. It is too weak because a generated expression does not carry a stable typed derivation explaining how each application, elimination, and visible specialization was justified. Leant therefore has to reconstruct that explanation from the original fragment and occurrence metadata.

4 Capability envelope and the limits that now matter

4.1 What is already unusually strong

It is easy to understate how far the combined system has moved beyond the classic Djinn and Exference projects. At the reviewed heads, the joint capability includes all of the following.

A checked, reusable neutral foundation. Names, types, kinds, constraints, declarations, environments, generated terms, queries, diagnostics, and search outcomes cross explicit validation edges. Both engines are available through one package and one public facade, while retaining independent search semantics.

Positive and negative search with honest evidence classes. Djinn supplies complete search and refutation within admitted projections; Exference supplies ranked positive search under budgets. Leant preserves that distinction and adds a source-language verification layer.

Bounded higher-rank and guarded impredicative synthesis. Positive forall occurrences can be opened according to a deterministic plan family. Hypothesis-side schemes, loaded schemes, query-local schemes, and provider schemes can be instantiated from carefully delimited vocabularies. Quantified choices are admitted only when supplied by the checked query or by explicit external evidence; the engines do not invent arbitrary polytypes.

Exact provider evidence. A richer frontend can preserve the correlation among all leading type binders selected by one instance head, including heterogeneous kinds. Evidence stays provider-local and is checked at the session boundary rather than being flattened into a global Cartesian pool.

Nominal and structural inductive identity. Compatible finite families can expose constructor introduction and elimination. Compatible recursive families have bounded positive construction in Djinn and one-layer elimination in Exference. Query-wide planning prevents separate occurrences of the same exact family from drifting into unrelated engine types.

Live source environments. Leant discovers a bounded, goal-relevant provider inventory from the actual Lean environment, preserves exact global names, accepts project ratings, caches semantic inventories, and verifies every use in the current session.

Source-faithful rendering of difficult applications. Visible type applications, mixed explicit and implicit forall binders, exact sort-domain alternatives, higher-kinded assignments, contextual rank-N arguments, and provider-result specialization all survive the round trip often enough to solve examples that a purely erased Haskell view could not express.

These capabilities should be treated as assets and compatibility constraints. A future redesign should not discard the deterministic prefixes, independent verification, fail-closed evidence checks, or public bounds that made the current system trustworthy.

4.2 Current finite bounds

Table 2 collects the most relevant finite limits at the reviewed heads. Some are public Djex constants; some are Leant policy values; and some are documented search-family bounds. Their exact values matter less than the fact that they are explicit, productive, and tested on malformed or cyclic caller-built inputs.

Table 2: Selected finite bounds at the review snapshot.

Bound	Value	Role
Provider instantiation candidates per checked query	32	Bounds provider-local scalar associations before search.
Exact provider assignments per checked query	32	Bounds complete correlated vectors accepted by public runners.
Leading arguments in one exact assignment	6	Single common eligibility boundary for current Djinn and Exference instantiation routes; seven is a fail-closed control.
Ground-kind constructor nodes per argument	129	Admits the complete right-associated `Type -> ... -> Type` shape through arity 64 while productively rejecting cyclic or excessive values.
Active Lean instance heads inspected per provider	32	Bounds frontend discovery in Lean resolver order.
Retained alpha-distinct vectors per live provider	16	Preserves correlated source choices without unbounded metadata growth.
Live providers serialized by Leant	80	Bounds one discovered inventory before staged search.
Semantic provider-cache entries	12	Generation-aware LRU keyed by canonical query/provider-world information.
Displayed candidates	5	Interactive result frontier.
Fresh groups tried per standalone engine	12	Backend verification budget after source-local deduplication.
Combined fresh-group frontier	24	Preserves twelve opportunities for each engine under the current merge order.
Engine candidate collection window	60	Bounds Djinn sorted collection and the shared ranking input.
Rank-N quintic plan views	512 per orientation	Covers all relevant ten- and eleven-site fifth-order selections; a twelve-site six/six split remains the documented gap.
Instantiation tuple attempts	512 per scheme family	Bounds fair candidate-tuple production.
Retained axioms	16 per scheme; 64 per family	Bounds positive-only generated specialization premises.

The right future direction is not to remove these bounds. It is to make each bound measure a semantic resource rather than a historical implementation dimension. For example, "at most 1,024 distinct compiled formula views" is a more general budget than "quintuple-open plus quintuple-opaque"; "at most 32 validated substitution certificates" is more general than a provider-only assignment vector; and "at most 60 semantic candidate fingerprints" is more useful than sixty derivations that later render to duplicates.

4.3 Five boundaries that block substantially more automation

4.3.1 The generated term is mostly untyped at the neutral edge

The search engines know types internally, and Leant knows the exact expected Lean fragment, but the shared generated expression is an unannotated syntax tree. Once a candidate crosses that edge, local type information, rule provenance, instantiation substitutions, and branch refinements are represented only indirectly through global maps and a re-fitting traversal.

This is the root cause behind several recent fixes:

- recovering a provider's specialized result from exact term arguments;

- carrying rank-N field types through case and let patterns;
- deciding where explicit forall placeholders must be woven;
- coupling a render-metadata choice to a specific provider occurrence;
- preserving trailing forall structure during higher-order application;
- distinguishing a canonical structural head from a merely similar nominal source fragment.

4.3.2 Quantifier occurrence planning is encoded by degree

The positive rank-N plan family has progressed from singleton choices through pairs, triples, quadruples, and capped quintuples. The implementation is carefully prefix-preserving and the coverage results are strong: the quintic family is exhaustive through eleven independent sites. But the next gap is already known: a twelve-site six-open/six-opaque split. Adding a sextic family would close that case and expose the next central antichain.

This is a sign that the representation of the search problem should change. The semantic object is a finite partially ordered set of occurrence decisions with ancestor prerequisites and formula equivalences. "Degree five" is one enumeration strategy for that object, not an enduring abstraction.

4.3.3 Source evidence is divided among specialized side channels

Current evidence channels include scalar provider candidates, exact assignment vectors, kinded vectors, provider binder names, exact forall-domain tags, contextual class fragments, nominal higher-kinded heads, constructor maps, type-name maps, and source fragments retained for fitting. Each addition has a reasonable local purpose. Together, they describe a more general object: a source-certified substitution or family fact with a stable provenance and a specified lifetime.

Until those channels share one certificate model, every new semantic fact must answer the same questions separately: how is it bounded, alpha-normalized, associated with a provider occurrence, invalidated with the session, projected into both engines, rendered in the source language, and rejected without poisoning unrelated evidence?

4.3.4 Dependent structure is transportable but not refinable

The current opaque-atom discipline is exactly right for many dependent goals. If two alpha-equivalent dependent propositions merely need to be transported, the engine can treat them as one atom and synthesize the higher-order plumbing. But constructor analysis of indexed families, equality transport, dependent pairs, subtypes, and motives remain beyond the structural projection.

A binary choice between "fully opaque" and "fully implement dependent type theory in the engine" would be a mistake. There is a valuable middle layer: search a structural term skeleton while retaining source-owned equality, index, universe, coercion, and type-class obligations as explicit holes.

4.3.5 Environment search is bounded by inventory width rather than semantic reachability

The provider-free baseline and geometric widths of 1, 4, 16, and the complete bounded inventory protect structural search from a large environment. They do not answer the harder retrieval question: which declarations are jointly useful for a type that requires two or three compositions, perhaps through higher-ranked or constrained intermediates?

A persistent Mathlib-scale inventory is already identified as future work in the project proposal. The key extension is not merely persistence. It is a semantic index and a backward relevance expansion driven by result types, argument requirements, family identities, constraints, and learned successful compositions.

5 Central diagnosis: the semantic recovery tax

5.1 A concrete model of the current round trip

A simplified current path for a difficult provider candidate is:

1. Lean elaborates the provider and goal.
2. The serializer projects them to `Frag`, preserving selected exact metadata in parallel records.
3. `Engine` lowers `Frag` into shared Djex types and private declarations, often erasing Lean binder roles or replacing exact families with collision-free engine names.
4. Djinn or Exference searches and emits a neutral `Expression` with visible type applications where the engine retained them.
5. `Render` reconstructs a local domain environment, strips synthetic premises, aliases provider occurrences, chooses source metadata, specializes provider results from exact arguments, fits nested term arguments, introduces missing binders, normalizes patterns, and enumerates textual variants.
6. Lean elaborates every variant against the original goal; failures are silently discarded.

The path is safe because step 6 is authoritative. Its incompleteness comes from steps 3–5: once information is erased, the renderer can recover it only when the original fragment and the generated syntax contain enough clues.

5.2 Why verification alone is not a substitute for typed candidates

One might argue that Lean is the checker, so the engine can emit arbitrary terms and let elaboration decide. That is correct for soundness but poor for search completeness and cost.

- **Rejected candidates consume the expensive resource.** The engine's pure search is often cheap relative to a backend round trip. Missing a type argument or choosing the wrong forall-domain spelling turns one semantic candidate into many elaboration attempts.
- **Elaboration failure is currently unstructured feedback.** A rejected spelling does not tell the engine whether the problem was a universe, binder visibility, an unsolved instance, a stale provider result, an index mismatch, or a genuinely wrong proof skeleton.
- **Untyped deduplication is weak.** Distinct derivations may normalize to the same source term; source variants of one semantic candidate may occupy multiple slots; and two terms with the same printed skeleton can differ in exact instantiation provenance.
- **Negative evidence cannot use verification failures.** A finite list of rejected candidates says nothing about inhabitation. The projection and search completeness must still be tracked independently.

5.3 The desired replacement

The replacement is not a fully source-specific core language. It is a *typed synthesis skeleton*:

- every term edge has an engine-visible expected or inferred neutral type;
- every local and global use has a stable occurrence identity;
- every explicit specialization records the substitution certificate that justified it;

- every elimination records the family descriptor and branch-local field types it exposes;
- facts that remain source-specific become named residual obligations;
- source rendering metadata is attached to the exact node whose choice it affects; and
- the candidate carries a derivation/provenance DAG sufficient for a small independent neutral checker.

Recommendation 1: introduce a typed candidate graph before adding another major type-language feature.

This is the highest-leverage architectural change. It directly simplifies provider-result fitting, occurrence-local metadata, structural elimination, semantic deduplication, elaboration diagnostics, and future dependent or recursive skeletons. It can be added incrementally beside the existing `Expression` output and projected back to that syntax for compatibility.

6 Proposed target architecture

6.1 Four layers with explicit contracts

The target architecture should make four layers visible.

Layer A: source semantic package.
Produced by the frontend after source elaboration. It contains a neutral type skeleton, opaque source atoms, exact family descriptors, provider certificates, binder roles, and bounded rendering metadata. In `Leant`, this package is produced by Lean metaprograms. In a Haskell frontend, it is produced from the checked source environment.

Layer B: logical/search projection.
A compact representation tailored to Djinn and Exference. It may erase universes, indices, dictionaries, and some quantifier occurrences, but it records every erasure as a structured projection-loss fact and retains stable links back to Layer A.

Layer C: typed candidate and derivation graph.
Search produces typed skeletons, not only surface expressions. Nodes refer to source/global/family identities through stable handles. Edges carry expected types, substitutions, and residual obligations. Both engines can project this graph to the current generated syntax.

Layer D: source completion and verification.
The frontend renders or directly builds a source AST, solves residual obligations using its elaborator, verifies the candidate, and optionally returns structured failure information to a bounded refinement loop.

The crucial rule is that no layer is asked to rediscover information owned by a previous layer. If a source fact is intentionally hidden from search, the candidate carries a reference or obligation, not an invitation for the renderer to infer it from text.

6.2 A frontend-neutral typed type skeleton

The existing shared `Type` should remain the Haskell-facing source type and the common type for the current engines. A new adjacent representation can express the additional distinctions required by richer frontends without forcing every engine to understand them.

One possible shape is:

Listing 3: Sketch of a richer neutral source-semantic type.

```
data SortClass
  = PropositionSort
  | DataSort
  | GeneralSort SortToken

data BinderRole
  = TypeBinder
  | TermBinder
  | InstanceBinder

data Visibility
  = Explicit
  | Implicit
  | StrictImplicit
  | InstanceImplicit

data SourceType variable atom
  = STVariable variable
  | STNominal FamilyId [SourceType variable atom]
  | STFunction (SourceType variable atom)
               (SourceType variable atom)
  | STPi (Binder variable atom)
         (SourceType variable atom)
  | STProduct ProductFlavor [SourceType variable atom]
  | STSum SumFlavor [SourceType variable atom]
  | STIndexed FamilyId
      [SourceType variable atom] -- parameters
      [atom]                     -- source-owned indices
  | STOpaque atom

data Binder variable atom = Binder
  { binderIdentity   :: BinderId
  , binderSourceName :: Maybe SourceName
  , binderRole       :: BinderRole
  , binderVisibility :: Visibility
  , binderSort       :: SortClass
  , binderDomain     :: SourceType variable atom
  }
```

This is deliberately not a complete Lean expression language. An index term or a universe expression may remain an opaque, hash-consed source token with frontend-provided alpha/definitional-equality relations. The search engines need only the structural distinctions that affect legal rules. A frontend can supply more exact data without making Djex depend on Lean.

The current `Frag` can initially be the Leant implementation of `SourceType`. The migration benefit comes from moving its reusable semantics—binder roles, exact family identities, occurrence IDs, and source obligations—to a public neutral package consumed directly by the typed candidate edge.

6.3 Typed candidate nodes and residual obligations

A candidate graph can be represented with typed node references rather than a recursive tree. Hash-consing then becomes natural, repeated subterms share storage, and provenance can refer to one stable node.

Listing 4: Sketch of a typed candidate envelope.

```
data TypedCandidate ty local source = TypedCandidate
  { candidateRoot        :: TermNodeId
  , candidateGraph       :: TermGraph ty local source
  , candidateObligations :: [Obligation ty source]
  , candidateDerivation  :: DerivationGraph
  , candidateLosses      :: ProjectionLossSet
  , candidateCost        :: CostVector
  , candidateFingerprint :: CandidateFingerprint
  }

data TermNode ty local source
  = LocalNode local ty
  | GlobalNode GlobalId ty source
  | LambdaNode [TypedPattern ty local] TermNodeId ty
  | ApplyNode TermNodeId TermNodeId ApplicationWitness ty
  | TypeApplyNode TermNodeId SubstitutionCertificateId ty
  | ConstructorNode ConstructorId [TermNodeId] ty
  | EliminateNode EliminatorId TermNodeId [BranchNode] ty
  | LetNode (TypedPattern ty local) TermNodeId TermNodeId ty
  | ObligationNode ObligationId ty

data Obligation ty source
  = TypeClassGoal source ty
  | EqualityGoal source ty ty
  | CoercionGoal source ty ty
  | UniverseGoal source
  | ElaborationChoice source [SourceAlternative]
  | TerminationGoal source
  | FrontendGoal source ty
```

`ApplicationWitness` need not contain a full proof object. It can record which neutral function type and substitution the engine used, enough for a small checker to re-run the typing step. `source` is an opaque frontend payload or handle, bounded and validated when the query is sealed.

This design makes the current `candidateResidualConstraints` a special case of `candidateObligations`. Ordinary Haskell type-class constraints remain directly representable. Lean instance synthesis, equality refinement, universe ambiguity, and coercion insertion become additional obligation kinds which Djex may carry without solving.

6.4 Stable occurrence identities

Recent Leant work had to create collision-safe aliases for separate uses of the same provider because two occurrences with the same canonical visible type may need different source visibility/domain alternatives. That is a strong argument for occurrence identity in the neutral graph.

Every source provider use, quantified occurrence, family occurrence, and opaque atom should have an `OccurrenceId` allocated by the checked frontend package. Search-created nodes may have separate internal IDs. The mapping must obey three invariants:

1. IDs are stable within one sealed query and never inferred from printed names.
2. Source metadata is keyed by the exact occurrence whose interpretation it changes.
3. Alpha-equivalent types may share a semantic fingerprint without losing distinct occurrence identity.

This removes the need to synthesize fake global names solely to index metadata, and it allows

multiple uses of one provider to make independent bounded choices while still normalizing to the same global source identity.

6.5 A typed, bounded source-certificate table

The current exact provider assignment is the prototype for a general source certificate. A useful abstraction is:

Listing 5: Generalized source-certified substitution evidence.

```
data SubstitutionCertificate ty source = SubstitutionCertificate
  { certificateOwner       :: EvidenceOwner
  , certificateTelescope   :: TelescopeFingerprint
  , certificateSubstitution:: [KindedArgument ty]
  , certificatePremises    :: [Obligation ty source]
  , certificateProvenance  :: source
  , certificateFidelity    :: FidelityWitness
  , certificateRendering   :: RenderingMetadata
  }

data EvidenceOwner
  = ProviderOwner GlobalId
  | LocalSchemeOwner OccurrenceId
  | FamilyOwner FamilyId
  | FrontendRuleOwner FrontendRuleId
```

The checked query boundary validates:

- owner/session identity;
- exact telescope length and binder order;
- kinds and closure;
- capture safety and alpha normalization;
- provenance-size bounds;
- structural fidelity where the engine relies on nominal distinctions;
- and source-local conflict handling.

Both engines then consume the same certificate, perhaps through different projections. Leant retains source rendering metadata under the certificate ID, not in a separate occurrence-guessing map. This generalization can cover active instance-head assignments, query-correlated choices, exact class contexts, family equalities, and future frontend-derived coercion or recursor facts.

6.6 Frontend-declared family algebra

The system needs a single language-neutral description of a structural family. Hard-coded recognition of functions, tuples, lists, `Either`, `Prod`, and `Sum` was a reasonable bootstrapping strategy. The recent higher-kinded structural-identity work shows why it should now be factored.

Listing 6: Sketch of a family descriptor supplied at the checked boundary.

```
data FamilyDescriptor ty source = FamilyDescriptor
  { familyId          :: FamilyId
  , familyKind        :: Kind
  , familySortClass   :: SortClass
  , familyParameters  :: [ParameterDescriptor]
```

```
  , familyIndices     :: [IndexDescriptor source]
  , familyConstructors:: [ConstructorDescriptor ty source]
  , familyEliminators :: [EliminatorDescriptor ty source]
  , familyRecursion   :: RecursionDescriptor
  , familyLaws        :: FamilyLawSet
  , familyRendering   :: source
  }

data ParameterRole
  = RepresentationalParameter
  | PhantomParameter
  | IndexDeterminingParameter
  | RecursiveParameter
```

The descriptor should state, rather than force the engines to rediscover:

- exact nominal identity and total arity;
- parameter versus index positions;
- the sort of the result;
- constructor field schemas after parameter substitution;
- whether elimination is finite, one-layer recursive, recursor-based, or unavailable;
- recursive SCC membership and strictly positive positions;
- whether a higher-kinded unsaturated head shares identity with an existing structural form;
- phantom/faithful parameter flow;
- and source rendering or AST handles.

For Haskell, the descriptor can be derived from checked data declarations and intrinsic tuple/function syntax. For Lean, it can be derived from `InductiveVal`, constructor telescopes, and recursor metadata. The engines still decide which subset they support. Unsupported laws are recorded as projection loss, not silently approximated.

This one abstraction would subsume a surprising amount of recent special-case work: query-wide family identity, total-arity planning, constructor maps, recursive occurrence descriptions, `Prod`/`Sum` structural identity, phantom-fidelity analysis, and source-specific constructor rendering.

6.7 Capability-indexed projection and evidence

The current `SynthRefuted Bool` is simple and safe, but the Boolean is becoming too coarse. A future outcome should explain what was complete and what was not.

Listing 7: Structured projection loss and negative evidence.

```
data ProjectionLoss source
  = OpaqueDependentAtom OccurrenceId source
  | BoundedQuantifierPlan [OccurrenceId]
  | UnsupportedFamilyElimination FamilyId
  | RecursiveDepthBound FamilyId Int
  | ProviderInventoryTruncated Int
  | ProviderEvidenceTruncated GlobalId
  | SearchBudgetReached SearchBudgetKind
  | FrontendDepthReached OccurrenceId
```

```
data NegativeEvidence source
  = RefutedInFragment CapabilityFingerprint RefutationCertificate
  | Inconclusive [ProjectionLoss source]
```

A refutation remains user-facing as a concise sentence, but internally it is indexed by a capability fingerprint: which connectives were exact, which families exposed complete constructor rules, whether every relevant forall occurrence was covered, whether provider premises were intentionally excluded, and whether any source atom hid analyzable structure.

This yields two benefits. First, new positive-only extensions can be added without threading another ad-hoc completeness flag. Second, a later frontend or engine can compare capability fingerprints and decide whether one refutation strictly dominates another.

Recommendation 2: generalize exact provider evidence and family metadata into checked source certificates.

Do this after the first typed-candidate slice, not before. Certificates give a uniform home to the information already flowing through provider assignments, rendering maps, family plans, and Lean-side evidence. They also preserve the projects' strongest invariant: untrusted caller-built metadata is validated at one finite boundary and cannot poison unrelated evidence.

7 Adaptive rank-N occurrence planning

7.1 Why another fixed-degree frontier is the wrong primary move

The current occurrence-plan history is disciplined:

- singleton frontiers established the basic positive-forall choice;
- pair, triple, and quadruple tails expanded complete coverage while preserving historical results;
- the quintic tail alternates selections from both source edges, caps each orientation at 512 compiled views, and covers every ten- and eleven-site fifth-order choice;
- explicit sentinels identify the next uncovered central split.

This is good engineering. It should remain as the deterministic compatibility prefix. But a sextic implementation would encode the same subset lattice one more time. The underlying problem can be searched directly.

7.2 Plan lattice

Give every positive forall occurrence a stable ID. A plan assigns each occurrence one of a small number of modes, initially:

$$\mathsf{Opaque}, \qquad \mathsf{Open}.$$

Nested occurrences impose prerequisite closure: opening a child requires opening the ancestor path that exposes it. Future modes may include $\mathsf{OpenWithCertificate}(c)$ or a source-specific obligation mode, but the binary lattice is enough for the first implementation.

A plan is normalized by:

1. ancestor closure;

2. elimination of choices for occurrences unreachable under the plan;
3. alpha-normalization of the resulting projected formula; and
4. hash-consing against previously compiled formula views.

Two distinct bit vectors that compile to the same formula consume one semantic plan slot, not two.

7.3 Compatibility prefix followed by best-first expansion

The scheduler should emit the current historical sequence unchanged through the quintic tails. If no term or conclusive result appears, continue with a best-first queue over normalized plans.

A practical priority can combine:

$$\begin{aligned}\mathsf{score}(p) = w_1\,\mathsf{newOpenSites}(p) + w_2\,\mathsf{projectedFormulaSize}(p)\\ + w_3\,\mathsf{estimatedPremiseCount}(p)\\ + w_4\,\mathsf{distanceFromFailedPlans}(p) - w_5\,\mathsf{queryCorrelation}(p).\end{aligned}$$

The exact weights are policy, not correctness. Determinism requires a stable tie-break by source occurrence order and normalized plan fingerprint.

Candidate successors can be generated by:

- flipping one boundary occurrence;
- adding the smallest unmet ancestor-closed set;
- moving toward the complement of a plan whose search was structurally unproductive;
- or satisfying a conflict set returned by the formula compiler or source verifier.

The budget should be stated in semantic units: maximum distinct compiled formula views, maximum cumulative formula nodes, maximum prepared-premise cache size, and maximum search work. The current 1,024 quintic views provide a reasonable starting ceiling for the adaptive tail.

7.4 Counterexample-guided refinement without trusting failures

Lean elaboration failures cannot justify negative evidence, but they can guide positive refinement. Suppose a candidate skeleton fails because one provider use requires an explicit type application whose source forall remained opaque. If the verifier returns a structured failure associated with occurrence o, the adaptive planner can prioritize plans opening o or select a certificate that fixes its source instantiation.

The feedback loop is heuristic only:

- a verifier rejection may increase a successor's priority;
- it never removes a plan needed for completeness;
- it never turns a bounded miss into a refutation; and
- all historical deterministic plans remain available independently of feedback quality.

Even without verifier feedback, the engine can derive useful conflict sets. Examples include a quantified hypothesis whose body cannot match any current goal subtree until a particular

occurrence is opened, or a compiled plan in which all newly exposed premises are duplicates of existing axioms.

7.5 Data structures

The first implementation does not require a sophisticated decision diagram. A persistent bit set plus a hash map keyed by compiled formula fingerprint is sufficient at current scales. If plans grow to dozens of sites, a zero-suppressed BDD is a natural representation for sparse occurrence subsets, while a normal BDD can compact complement-related open/opaque families. The key is to keep that optimization below the semantic interface.

Recommendation 3: preserve the current quintic prefix, then replace degree-by-degree widening with an adaptive normalized plan lattice.

This closes the known twelve-site central gap without committing the project to a permanent sequence of sextic, septic, and higher frontiers. It also creates a place to use structured verification feedback and query-correlation scores while retaining finite, deterministic search.

8 Shared subgoal DAG, semantic identity, and dominance

8.1 Why both engines should share more than input declarations

The current combined mode runs Djinn and Exference independently and then merges candidate groups with a carefully designed fairness schedule. This preserves each engine's semantics, but it duplicates work at three levels:

- equivalent type substitutions and opened-forall views are prepared separately;
- structurally identical subgoals are explored under independently built environments; and
- semantically duplicate candidates are often recognized only after rendering or exact-spelling comparison.

The engines need not share one search algorithm to share immutable canonical objects. A neutral query preparation layer can intern types, substitutions, family descriptors, local contexts, and subgoal keys. Each engine then keeps its own frontier and proof theory while reusing the same identities and caches.

8.2 Canonical subgoal keys

A subgoal cache key should include at least:

$$\begin{aligned} K = (&\text{goal type},\ \text{local-environment fingerprint},\\ &\text{rigid scope},\ \text{available certificates},\\ &\text{usage policy},\ \text{projection capability}). \end{aligned}$$

The local environment fingerprint must be order-sensitive where the engine's enumeration order matters, but it can separately carry an order-insensitive logical fingerprint for cache reuse. This distinction mirrors the existing oldest-first fairness work: proof identity and candidate ordering are related but not identical concerns.

Prepared premise families, rank-N plan views, and exact provider substitutions can all reference the same interned type nodes. The current prepared-premise caches in Djinn are a natural starting point.

8.3 Semantic candidate fingerprints

A candidate fingerprint should be computed before source rendering. It must preserve semantic distinctions that affect typing while eliminating irrelevant implementation history.

Recommended normalization steps are:

1. alpha-normalize local binders and source-owned closed type binders;
2. flatten and rebuild mixed application spines in canonical source order;
3. normalize explicit constructor identities through family descriptors;
4. perform safe beta reduction of administrative lambdas and let nodes;
5. eta-contract only when the typed graph proves the contraction preserves explicit source binder obligations;
6. canonicalize irrefutable tuple patterns and branch ordering where the family descriptor permits it;
7. retain certificate IDs only through their semantic substitution and obligation fingerprint, not allocation order; and
8. include residual obligations and expected result type.

The result is not necessarily a source-language normal form. It is a stable identity for scheduling and deduplication. Source renderers may still offer multiple spellings of one fingerprint.

This change would improve several current limits immediately. The sixty-item collection window would contain sixty semantic terms rather than sixty backend derivations. The twelve-group verification frontier would not be spent on terms already rejected under the same source semantics. Combined mode could merge by fingerprint before applying its $D_1 \dots D_4, E_1 \dots E_{12}, D_5 \dots D_{12}$ fairness schedule.

8.4 Dominance pruning

For one semantic term, prefer a candidate that:

- has fewer or weaker residual obligations;
- relies on a smaller projection-loss set;
- uses fewer providers or lower total provider penalty;
- has a smaller typed term graph;
- needs fewer source rendering alternatives; and
- preserves more exact source metadata.

This defines a partial order rather than one scalar score. Keep the Pareto frontier per semantic fingerprint. A source frontend may accept a slightly larger term because it has no unsolved typeclass obligation, while a Haskell frontend may prefer the opposite. The current engine-specific details can remain as tie-break data below this neutral dominance relation.

8.5 Optional equality saturation

An e-graph is not required for the first implementation, but the typed term DAG makes it possible later. A small, explicitly typed rewrite set could identify administrative equivalences such as:

- beta/eta and let-floating within capture-safe scopes;
- tuple projection after known construction;
- case-of-constructor simplification;
- reassociation of engine-generated applications; and
- elimination of synthetic premise lambdas after substitution.

The rewrite set must never use proof irrelevance or source-specific definitional equality unless the frontend supplies an exact certificate. Equality saturation is therefore an optimization for candidate identity and simplification, not a replacement for source checking.

Recommendation 4: deduplicate and rank typed semantic candidates before rendering.

This is likely the best near-term latency improvement after the typed edge. It reduces expensive Lean verification, improves the meaning of existing windows, and gives combined mode a shared unit of fairness without merging the engines' search semantics.

9 First-class evidence and source obligations

9.1 From erased dictionaries to explicit proof goals

The present architecture deliberately avoids treating Lean instance binders as ordinary proof power inside Djinn. A provider use can leave a class argument implicit and let Lean's elaborator rebuild it. Exact assignment discovery can use active instance heads to determine type arguments, but the dictionary itself is not transported as a general engine value.

That conservative boundary is sound and has avoided cross-language confusion. It also excludes useful terms whose structure depends on an available class method or whose result has residual evidence obligations.

A safe extension is not to import Lean dictionaries as executable Haskell terms. It is to let the candidate graph contain an opaque evidence node:

```
EvidenceNode
  { evidenceGoal       :: SourceEvidenceGoalId
  , evidenceNeutralType:: NeutralConstraint
  , evidenceExpected   :: Type
  }
```

The frontend certifies that the goal is well-formed in the sealed environment. The engine can carry the node, compose providers around it, and charge it a cost. The source completion layer asks Lean's instance synthesizer to solve it in the exact local context. Failure rejects the candidate; success fills the hole with a source expression not interpreted by Djex.

This supports a useful middle ground:

- class-constrained provider composition;
- `Decidable` and coercion obligations;

- higher-kinded constraints retained inside rank-N arguments;
- local instance hypotheses in prove mode; and
- source-specific elaboration mechanisms that have no meaningful Haskell analogue.

9.2 Evidence tables rather than arbitrary callbacks

For determinism, purity, and cacheability, the preferred first design is a bounded evidence table produced before search, not an arbitrary engine-to-Lean callback during lazy evaluation. A frontend request can include:

- exact evidence goals known to be solvable in the current environment;
- the type variables or source metavariables they determine;
- complete correlated substitutions, where relevant;
- a stable provenance token and environment generation; and
- a cost or priority supplied by source resolution order.

Search may use only table entries. A later bounded refinement round may ask the frontend for additional evidence goals discovered in a candidate skeleton, but each round seals a new immutable request.

This reproduces the strongest properties of current active-instance assignments: isolation between attempts, exact vector correlation, finite limits, and no mutation of the next candidate's metavariable state.

9.3 Equality and coercion obligations

The same mechanism can carry source equalities without teaching the engines a full dependent conversion checker. An `EqualityGoal` contains opaque source terms and their neutral types. Search rules may create it only through a family descriptor or frontend certificate. The source layer discharges it using definitional equality, an available proof, a coercion, or an explicit transport term.

For example, a constructor branch of an indexed family may refine an index from n to 0 or $m+1$. The engine can solve the structural branch body under a source refinement token while leaving the exact equality proof to Lean. The token is scoped to that branch and cannot escape unless the candidate carries a corresponding equality obligation.

9.4 Obligation-aware cost

Residual obligations should be visible to ranking. A reasonable default cost order is:

$$\begin{aligned}\text{no obligation} &< \text{known table evidence} < \text{ordinary instance goal} \\ &< \text{definitional equality} < \text{explicit equality transport} \\ &< \text{open frontend hole}.\end{aligned}$$

The order is policy and frontend-dependent. The important invariant is that a term with unresolved obligations is never presented as complete before the source layer fills and checks them.

10 Recursive synthesis through recursor skeletons

10.1 Current conservative recursive support

The current split is principled:

- Djinn can construct bounded positive recursive values, limited by SCC-aware rules and an independent-SCC cap;
- Exference can perform one finite constructor match and treats recursive fields as ordinary locals without recursively eliminating them;
- neither engine invents an unbounded recursive definition or performs induction; and
- Lean or GHC remains the final checker of generated terms.

This covers finite constructor terms and useful one-layer projections while keeping termination obvious. The next step should preserve that posture.

10.2 Recursors as ordinary certified providers

In Lean, each inductive family already has recursors and cases eliminators with checked termination. Rather than adding general recursion to the engine, serialize a bounded *recursor descriptor* as part of the family algebra. The descriptor identifies:

- the motive parameters and indices;
- the major premise;
- one branch telescope per constructor;
- which branch fields receive recursive hypotheses;
- universe/sort restrictions; and
- a source handle for rendering the exact recursor application.

Search can then synthesize a recursor skeleton:

$$\mathsf{recursor\ motive\ branch}_1\ \cdots\ \mathsf{branch}_k\ \mathsf{major}.$$

Each branch is a typed subgoal. Recursive hypotheses are ordinary local providers whose types come from the descriptor. Djinn or Exference can solve branch plumbing without ever constructing a self-reference.

For Haskell, the analogous descriptor can point at an existing trusted fold or eliminator such as `foldr`, rather than requiring the neutral term syntax to express `let rec`. A future Haskell-specific frontend may add checked structural recursion separately.

10.3 When to propose a recursor

Unrestricted recursor insertion can explode the search space. Gate it by source and goal relevance:

1. a local or provider value has an exact recursive family at its head;
2. the target type depends on, or is reachable from, fields exposed by one constructor layer;
3. one-layer case analysis failed or leaves a branch goal whose type matches the recursive result at a structurally smaller field;
4. a recursor descriptor is complete and source-certified; and
5. the per-query recursor budget has not been spent.

Start with non-dependent catamorphism-like recursors for `Nat`, `List`, and simple user families. The branch result type must be uniform and first-order in the initial slice. Later, the same typed skeleton can carry motive and index obligations.

10.4 Bounded recursion policies

Useful finite controls include:

- at most one recursor skeleton on a term path initially;
- at most one recursive family SCC per skeleton;
- a cap on branch subgoals and recursive-hypothesis uses;
- no synthesized nested recursor until all one-recursor candidates have been tried;
- a size penalty dominating ordinary application and one-layer cases;
- and no negative completeness claim for projections that omit an available recursor.

These bounds are semantic and can be widened independently of the source language. Lean's termination checker remains authoritative because the neutral engine never generates an unchecked recursive binding.

Recommendation 5: add certified recursor applications before general recursive definitions or induction tactics.

Recursor skeletons substantially widen useful program and proof synthesis while preserving the current termination story. They reuse family descriptors, typed branch goals, provider premises, and source obligations introduced by the preceding architectural work.

11 A staged dependent and indexed fragment

11.1 Why a staged approach is feasible

Full dependent program synthesis is not a single feature. It combines higher-order unification, definitional equality, index refinement, motive synthesis, proof search, coercions, and source elaboration. Attempting all of that in Djex would destroy the present division of responsibility.

The typed-skeleton architecture permits several useful fragments in increasing order of difficulty.

11.2 Stage D0: exact dependent transport

This stage largely exists already through alpha-aware opaque atoms. Make it explicit and more reliable:

- hash-cons dependent source atoms by an exact frontend fingerprint;
- distinguish definitional equality, alpha equality, and mere pretty-text equality;
- preserve occurrence IDs and binder scopes;
- let structural rules transport, pair, inject, select, and compose these atoms without inspecting their internals; and
- report hidden dependent atoms in projection-loss diagnostics.

This improves higher-order plumbing over propositions such as $\forall n, P\, n$ without adding dependent elimination.

11.3 Stage D1: dependent pairs and subtypes as certified structures

`Sigma`, dependent records, and `Subtype` can be represented by family descriptors whose later field types refer to earlier branch locals via opaque source tokens. The engine need not substitute those source terms itself. It needs typed pattern binders and a scoped obligation environment.

A constructor skeleton for $\Sigma x : A, B\,x$ has two subgoals:

$$a : A, \qquad b : B\,a.$$

The second expected type is a source handle instantiated by the frontend after a is chosen. In the initial implementation, require a to be an exact local/global term known to the frontend, not an arbitrary generated expression. This still enables useful packaging and projection terms.

Elimination exposes the first component and a second component whose exact type is supplied by the branch descriptor. Lean verifies the dependency.

11.4 Stage D2: indexed constructor refinement

For an indexed inductive, constructor matching refines indices. A descriptor can attach branch-local equations and specialized field types to each constructor. Search proceeds structurally under a `RefinementContext`:

```
data RefinementContext source = RefinementContext
  { refinedIndices :: [(source, source)]
  , localEqualities:: [ObligationId]
  , sourceScope    :: SourceScopeId
  }
```

The neutral checker ensures that a branch cannot use a refinement token outside its scope. The frontend decides whether refinements are definitional, require an equality proof, or need an explicit transport. A failed source check drops the candidate.

Good initial targets include:

- equality itself, with reflexivity and one controlled transport rule;
- `Fin` and simple zero/successor index splits;
- vectors under one constructor match; and
- user GADTs whose branch result indices are syntactically constructor forms after elaboration.

11.5 Stage D3: motive holes and recursors

Only after D1 and D2 should the engine generate dependent recursor skeletons. The motive is an explicit frontend obligation with a neutral shape and exact source expected sort. Search can use common templates:

- constant motive;
- target-as-motive after abstracting the major premise;
- one index-variable abstraction; and
- a frontend-supplied finite motive candidate table.

Lean elaboration determines universe correctness and dependent conversion. No negative completeness claim should cover an omitted motive template.

11.6 What should remain opaque

Even after these stages, the following should remain source-owned unless a specific checked rule is added:

- arbitrary term-level computation inside indices;
- unrestricted higher-order unification;
- synthesis of arbitrary equality lemmas;
- quotient and heterogeneous equality reasoning;
- well-founded recursion and termination measures;
- and source-specific coercion search beyond finite obligations.

This boundary is not a failure. It lets the system automate a much larger class of complex types while preserving an understandable and testable search core.

12 Semantic environment retrieval at project and Mathlib scale

12.1 Separate retrieval from search

The current live-provider inventory is intentionally bounded and goal-relevant, with project ratings and a semantic cache. This validates the basic source integration. Scaling it should not mean handing thousands of declarations to Exference and hoping its ranking compensates.

Introduce a distinct *retrieval layer*. Its output is still a finite, checked provider package, so the existing engine boundary and soundness model remain intact. Retrieval can evolve independently and can be tested against known useful provider sets.

12.2 Persistent signature index

For each declaration, store a compact normalized signature record:

```
data ProviderIndexEntry = ProviderIndexEntry
  { providerId          :: GlobalId
  , providerResultHeads :: [HeadFeature]
  , providerArgumentHeads:: [HeadFeature]
  , providerForallShape :: ForallShape
  , providerArrowDepth  :: Int
  , providerFamilyUses  :: [FamilyId]
  , providerConstraints :: [ConstraintFeature]
  , providerSortClass   :: SortClass
  , providerTypeSize    :: Int
  , providerNamespace   :: NamespaceFeature
  , providerModule      :: ModuleId
  , providerBaseRating  :: Rating
  , providerFingerprint :: TypeFingerprint
  }
```

A `HeadFeature` should distinguish exact family identity from opaque nominal text. Include partial higher-kinded heads and structural family IDs, so a query for `Prod` can retrieve declarations indexed under the same family descriptor as saturated products.

In `Leant`, the index can live in the synthesis companion already associated with snapshots, or in a project cache keyed by Lean toolchain, imported module fingerprints, and relevant options.

Incremental updates should append entries for session declarations and invalidate only affected namespace/result-head buckets.

12.3 Backward type-reachability expansion

Start from the exact goal feature set. Retrieve providers whose result can match the goal under cheap head/kind/arity filters. Their unsolved argument features become the next frontier. Continue for a small semantic radius:

$$G_0 = \mathsf{features}(\text{goal}), \qquad G_{i+1} = G_i \cup \bigcup_{p:\mathsf{result}(p)\rightsquigarrow G_i} \mathsf{arguments}(p).$$

This is not full unification. It is an admissible retrieval over-approximation. The checked engine still validates and searches exact types. Useful filters include:

- exact result head and structural family identity;
- result-arrow depth;
- proper kind and sort compatibility;
- quantified-result shape;
- required local head features already present in the query;
- constraint satisfiability from the bounded evidence table;
- namespace/module proximity;
- and user ratings.

At each radius, apply diversity quotas: do not let dozens of near-identical lemmas from one namespace crowd out a composition bridge. Keep at least a few entries per distinct result fingerprint, argument-head profile, and module.

12.4 Proof-graph retrieval

Successful synthesized terms provide valuable but deterministic local data. Record a graph edge from a provider's result feature to each argument feature, weighted by successful use count, final candidate rank, and verification success. Project-local learning can reorder retrieval while preserving an unchanged lexical fallback and explicit ratings.

This is not machine learning in the opaque-model sense. It is a transparent weighted dependency graph. Its cache can be deleted without changing soundness or available rules; only ranking changes.

12.5 Inventory packages and staged widening

Replace raw widths $1, 4, 16, 80$ with semantic packages:

1. exact result-head providers;
2. one-step bridge providers;
3. constraint/evidence-complete providers;
4. two-step backward-reachable providers;
5. namespace/project neighborhood;

6. and the full bounded retrieved set.

The current geometric widths can remain as caps within each package during migration. The important change is that every stage has a semantic reason and can be cached by a query feature fingerprint.

Recommendation 6: build a persistent semantic index and backward relevance expander before raising the live-provider cap.

This is the path to useful Mathlib-scale synthesis. A larger unstructured inventory would mostly increase queue pressure, duplicate candidates, and Lean verification cost; a semantic retrieval layer increases the probability that a small inventory contains the necessary composition chain.

13 Cooperative Djinn–Exference search

13.1 Keep independent semantics, share typed artifacts

A single hybrid engine would risk losing the clearest property of the current system: Djinn has a complete logical interpretation for an admitted fragment; Exference has heuristic ranked search and evidence resolution. The better design is a portfolio over shared typed artifacts.

Three levels of cooperation are increasingly powerful.

13.1.1 Level 1: shared preparation and semantic caches

Both engines consume the same interned types, source certificates, family descriptors, normalized provider inventory, and plan fingerprints. They share candidate fingerprints and verification outcomes. Search frontiers remain independent.

This is low risk and should accompany the typed candidate graph.

13.1.2 Level 2: progress-aware portfolio scheduling

Replace fixed batch quotas after the compatibility prefix with a deterministic portfolio scheduler. Give the next slice of work to the engine with the best recent marginal progress, measured by:

- new semantic candidates produced;
- new subgoal fingerprints solved;
- reduction in unsolved obligation count;
- source-verification acceptance rate;
- unique family/provider coverage;
- and frontier growth per unit of engine work.

Retain minimum service guarantees: each standalone twelve-group opportunity must still be reachable, and a complete Djinn refutation cannot be delayed indefinitely by an expanding Exference queue. Deterministic counters and stable tie-breaking preserve reproducibility.

13.1.3 Level 3: proof skeletons and leaf filling

The typed candidate graph enables a more interesting composition:

1. Djinn produces a structural derivation with one or more opaque atomic/provider/evidence leaves.
2. Each leaf becomes a typed subquery for Exference with the exact local context and obligations.

3. Exference fills leaves using live providers, class evidence, or recursive-family rules.

4. The combined graph is checked by the neutral checker and the source frontend.

Conversely, an Exference candidate can contain structural holes which Djinn solves completely. This is especially promising for environment-rich higher-order plumbing: Exference chooses the right library bridge, while Djinn fills the pure structural lambda surrounding it.

The leaf protocol must prevent cyclic self-justification. A subquery records its parent candidate fingerprint and forbids a provider/certificate dependency cycle unless the source family descriptor explicitly authorizes a recursor.

13.2 Refutations in a cooperative portfolio

Only a complete engine projection may refute its exact subgoal. A filled candidate can use such refutations to prune a branch locally, but the overall query is refuted only when:

- the root projection is complete;
- every structural plan required by the capability fingerprint has been exhausted;
- no omitted provider or source obligation could add proof power; and
- every cooperative leaf is either solved or refuted in a complete subfragment.

In practice, most new cooperative modes will be positive-only at first. Their presence simply adds a `ProjectionLoss` to any otherwise complete provider-free refutation. This is the conservative rule already used by many current extensions, generalized structurally.

14 Concrete changes by repository

14.1 Djex responsibilities

The following additions belong naturally in Djex because they are frontend-neutral and should be shared by both engines.

New neutral modules.

- `Language.Haskell.Synthesis.TypedGenerated`: typed term graph, patterns, application witnesses, obligations, and fingerprints;
- `Language.Haskell.Synthesis.Certificate`: checked source certificates, ownership, provenance bounds, and fidelity witnesses;
- `Language.Haskell.Synthesis.Family`: family/constructor/eliminator descriptors and recursion metadata;
- `Language.Haskell.Synthesis.Capability`: projection losses, capability fingerprints, and structured negative evidence;
- `Language.Haskell.Synthesis.Plan`: occurrence IDs, normalized plan lattice, deterministic adaptive scheduler, and plan fingerprints;
- `Language.Haskell.Synthesis.Fingerprint`: interned type, context, subgoal, and candidate identities; and
- optionally `Language.Haskell.Djex.Portfolio`: shared preparation and progress-aware orchestration without merging engine internals.

Backend adapters. Each engine adapter should produce a typed candidate derivation. This need not rewrite the engines immediately. The first version can instrument existing search/checker steps:

- Djinn maps proof-term constructors and formula nodes to typed graph nodes while erasing synthetic instantiation premises through explicit substitution witnesses;
- Exference maps holes, inventory uses, unification substitutions, tuple/constructor steps, and case rules to the same graph;
- the current `Expression` is generated from the typed graph by a compatibility projection; and
- a small neutral checker validates every candidate before it leaves Djex.

Public query API. Add a versioned request extension rather than changing every existing field:

```
data QuerySemanticPackage variable source = QuerySemanticPackage
  { semanticFamilies     :: [FamilyDescriptor ...]
  , semanticCertificates :: [SubstitutionCertificate ...]
  , semanticOccurrences  :: OccurrenceTable source
  , semanticCapabilities :: RequestedCapabilities
  , semanticLimits       :: SemanticLimits
  }
```

Legacy callers receive an empty package derived from existing declarations and provider evidence. This preserves the curated public facade and permits a long migration.

Limits. Keep the current public constants as compatibility defaults. Introduce a `SemanticLimits` record whose fields are measured in unique plan views, certificate nodes, typed graph nodes, obligation count, and fingerprint cache size. Reject malformed lazy/cyclic values at sealing time exactly as current kind and assignment bounds do.

14.2 Leant responsibilities

Leant should remain the authority for elaborated Lean semantics.

Source package producer. Extend the generated Lean metaprogram to emit:

- stable occurrence IDs for every serialized goal/provider/family node;
- exact binder roles, visibility, and sort-domain tokens;
- family descriptors derived from Lean inductive and recursor metadata;
- active-instance and other substitution certificates;
- exact source AST handles or bounded structural rendering metadata;
- opaque term/index fingerprints with source equality classes; and
- an environment generation/fingerprint for cache invalidation.

The producer already computes much of this information in specialized forms. The change is to emit one typed package rather than several parser-position specific strings and maps.

Renderer becomes source completion. `Leant.Synth.Render` should gradually stop inferring types from untyped syntax. Its core operation becomes:

1. traverse a typed candidate graph;
2. look up exact source metadata by occurrence/certificate/family ID;
3. materialize Lean syntax or an AST command;
4. discharge obligations in a bounded source-completion phase;
5. elaborate against the exact goal; and
6. return either a verified term or a structured rejection.

The existing fitting code remains as a compatibility path for legacy `Expression` candidates until all engines emit enough typed data. Regression tests should assert that new candidates use the typed path and do not silently fall back.

Verification API. Extend the backend worker response with a small error taxonomy:

```
data VerificationFailure
  = TypeMismatch OccurrenceId SourceTypeSummary SourceTypeSummary
  | UnsolvedInstance OccurrenceId SourceGoalSummary
  | UnsolvedMetavariable OccurrenceId
  | UniverseMismatch OccurrenceId
  | BinderVisibilityMismatch OccurrenceId
  | UnknownSourceIdentity SourceIdentity
  | IndexRefinementFailure OccurrenceId
  | TerminationFailure OccurrenceId
  | OtherElaborationFailure String
```

This information is diagnostic and heuristic. The full Lean message can still be retained for debugging. The engine never trusts the category for soundness.

Provider index. Add a persistent index builder and query feature extractor outside `Leant.Synth.Engine`. The engine continues to receive a finite checked inventory. Provider cache keys incorporate the index generation and semantic retrieval stage rather than only a width.

14.3 Shared testing responsibilities

Every cross-repository capability should continue the current paired pattern:

1. neutral Djex unit/property tests;
2. public-facade tests through both engines;
3. generated Haskell accepted by GHC where applicable;
4. Leant pure serializer/adapter/renderer tests;
5. malformed and over-bound controls;
6. a Lean transcript covering Djinn, Exference, and combined mode;
7. and a commit in Leant pinning the exact reviewed Djex revision.

The typed architecture adds two valuable differential checks: the neutral checker must accept every engine candidate, and the source verifier must accept every displayed candidate. A mismatch between those layers is a localized boundary bug rather than an unexplained text-rendering failure.

15 Worked target classes

This section illustrates what the proposed architecture would make possible. The examples are not promises that one generic algorithm solves all of them. They identify the smallest new semantic mechanism needed beyond the current system.

15.1 Higher-rank provider composition with exact source metadata

Consider a schematic provider family:

```
select
  :: forall f g.
     Evidence f g
  => Box (f A B)
  -> (forall x. f x x -> g x)
  -> Result (g A)
```

and a source environment establishing one correlated choice $f := (,)$, $g := \mathsf{Either}\ E$, with a contextual rank-N argument. The recent structural higher-kinded work can preserve closely related cases, but the complete path currently relies on parallel canonical/source representations and renderer fitting.

Under the proposed architecture:

1. the frontend emits one substitution certificate for `select`;
2. the certificate refers to the canonical pair/sum family descriptors and carries exact Lean rendering metadata;
3. the typed application node records the selected substitution and the specialized type of each term argument;
4. any class evidence is a source obligation associated with the same certificate; and
5. the renderer performs no type reconstruction—it materializes the known specialization and asks Lean to discharge the evidence.

This generalizes the latest `Prod`/`Sum` fixes to arbitrary families whose descriptors explicitly declare an unsaturated structural identity.

15.2 A fold-shaped list program

A target such as

$$(\alpha \to \beta \to \beta) \to \beta \to \mathsf{List}\,\alpha \to \beta$$

has the obvious `List.foldr`-shaped inhabitant. Current live-provider search may find the library function if it is retrieved. A source-certified recursor descriptor allows synthesis even without a provider indexed under the exact target spelling:

1. choose the `List` recursor because the major premise is a list and the target result is uniform;

2. create nil- and cons-branch subgoals;
3. expose the recursive result as a typed local in the cons branch;
4. let Djinn synthesize the structural application of the user's step function, or let Exference choose a relevant provider; and
5. render an exact `List.rec`, `List.foldr`, or source-preferred eliminator application supplied by the descriptor.

No recursive binding is generated, and Lean's recursor is already termination checked.

15.3 A dependent pair projection

For

$$(\Sigma x : A, B\,x) \to A,$$

the engine needs only the first field type. A dependent-family descriptor makes the pair eliminable; the second field remains source-typed under the first binder but can be unused. The typed case node exposes $x : A$ and $y : B\,x$; Djinn returns x. No equality reasoning or motive synthesis is required.

For

$$(\Sigma x : A, B\,x) \to \Sigma x : A, B\,x,$$

the current opaque-transport route can already return the input. The new descriptor merely gives the engine an additional constructor/eliminator view without sacrificing the exact opaque identity.

15.4 One-layer indexed refinement

For a vector-like family, consider a function which eliminates an impossible constructor at length zero. The source descriptor says that the nil constructor returns index zero and cons returns successor. In a branch where the major premise has index zero:

- the nil branch is admitted directly;
- the cons branch carries an equality obligation $\mathsf{succ}(n) = 0$;
- Lean can discharge it as impossible during elaboration or via a source no-confusion rule; and
- the candidate graph records why the branch is impossible rather than treating the whole indexed family as unrelated atoms.

This is a bounded constructor-refinement rule, not general dependent proof search.

15.5 Class-constrained higher-order plumbing

Suppose the environment contains:

```
mapMLike :: Monad m => (a -> m b) -> Container a -> m (Container b)
```

and the target supplies a function `a -> m b` and a container. The engine can structurally choose and apply the provider while leaving `Monad m` as an evidence obligation. In Leant, the frontend asks type-class synthesis to fill it in the exact context. If no instance exists, the candidate is

rejected or retained as a visibly incomplete skeleton in a future diagnostic mode; it is never displayed as a finished term.

This is more expressive than erasing dictionaries entirely, but safer and more portable than importing source dictionary terms into the neutral engine.

15.6 Library bridge plus structural completion

A common complex synthesis shape is:

$$\text{local higher-order plumbing} \circ \text{environment provider(s)} \circ \text{local constructor/recursor plumbing.}$$

A cooperative portfolio can solve this efficiently:

1. semantic retrieval selects a provider whose result head reaches the goal;
2. Exference chooses that bridge and leaves typed argument holes;
3. Djinn fills purely structural holes completely;
4. source evidence fills class/equality obligations; and
5. semantic normalization removes administrative differences before Lean verification.

This is the most plausible route to substantially more automation for complex types. It uses each subsystem where it is strongest instead of forcing one engine to solve the entire problem monolithically.

16 Soundness, completeness, and trust boundaries

16.1 Positive soundness remains source-verified

The fundamental positive-soundness theorem is operational:

> A candidate is presented as a term of source type T only after the source frontend has elaborated and checked that exact candidate against T in the current sealed environment.

The typed neutral checker is defense in depth and a debugging aid. It catches engine/boundary inconsistencies earlier, permits semantic normalization, and makes source failures more meaningful. It does not replace GHC or Lean.

Residual obligations preserve this theorem. A candidate with unfilled obligations is a skeleton, not a finished term. The source completion layer must fill every obligation and re-check the result before display.

16.2 Certificate trust is bounded and checked

A source certificate is not trusted merely because it came from a frontend. The sealed query validates the structural facts on which search relies:

- no duplicate or stale owner identity;
- exact telescope fingerprint and argument count;
- well-formed kinds and closed substitutions;
- bounded source payload size;

- no free source variables outside the declared scope;
- no conflicting family arities or class kind vectors;
- and required fidelity through datatype fields when nominal identity is used to justify a visible specialization.

Source-specific claims that Djex cannot validate are obligations, not search axioms. For example, "Lean can synthesize this instance" is confirmed by Lean during source completion; it is not used to prove a Djinn refutation.

16.3 Negative evidence must be monotone under capability loss

Let P be the exact source problem and $\pi_C(P)$ a logical projection under capability set C. A Djinn refutation is source-valid only if the projection is complete for the stated fragment. Adding positive-only certificates or omitted source rules must not strengthen that claim.

A useful invariant is monotonicity:

$$C_1 \subseteq C_2 \quad \Longrightarrow \quad \mathsf{loss}(C_2) \subseteq \mathsf{loss}(C_1).$$

A refutation at C_1 can be reused at C_2 only when the newly added capabilities introduce no new proof power relevant to the root problem, or when those capabilities are themselves exhaustively searched. Otherwise the old refutation is retained only as an internal provider-free fallback, exactly as Leant already does.

16.4 Refutation certificates

A future Djinn adapter could emit a compact certificate of saturated LJT search rather than only a status. The certificate need not be a proof term of negation; it can be a replayable finite derivation showing that every applicable rule branch closes unsuccessfully under the normalized formula and context.

Benefits include:

- independent checking of negative results;
- stable caching keyed by formula and capability fingerprint;
- clearer diagnostics for which atom or omitted rule prevents a source refutation; and
- safer cooperative pruning of typed subgoals.

This is lower priority than typed positive candidates because the current Djinn boundary already has a strong completeness discipline. It becomes more valuable once capabilities and cooperative subqueries are more numerous.

16.5 No semantic inference from source spelling

The current projects already move away from raw text: validated names, alpha-normalized closed types, exact family keys, and structural context payloads replace guessed strings. The target architecture should make the rule absolute:

> Source spelling is for presentation and source AST construction. Search identity, kinds, family arity, binder roles, and evidence ownership come only from checked structural data.

This is particularly important for Lean, where pretty printing can omit universes, implicit arguments, qualifications, coercions, and binder information.

17 Implementation roadmap

The roadmap below is dependency-ordered. Effort labels are relative architectural sizes, not calendar estimates.

17.1 Milestone 0: observability and baseline corpus (S/M)

Before changing the candidate edge, make current costs visible.

- Add per-query counters for compiled rank-N plans, unique formula fingerprints, prepared-premise cache hits, engine states, raw derivations, rendered groups, semantic/exact duplicates, Lean variants tried, and verification failure classes.
- Record provider retrieval stage, inventory composition, and provider use in each verified candidate.
- Store machine-readable benchmark output beside the human transcripts.
- Freeze a representative corpus from the current pure tests and Lean goldens, including every known fail-closed sentinel.

This milestone produces an immediate result: it will reveal whether current latency is dominated by plan compilation, Exference queue growth, renderer variant multiplication, duplicate candidate groups, or Lean round trips for each class of query.

17.2 Milestone 1: typed candidate spine and occurrence IDs (M/L)

Implement the smallest typed slice that eliminates recent reconstruction work:

- stable source occurrence IDs;
- typed local/global/application/type-application/tuple/let/case nodes;
- application witnesses containing the exact neutral function domain and result;
- provider certificate IDs on visible specializations;
- typed pattern fields for product and finite-family elimination;
- a neutral checker; and
- projection back to the existing `Expression`.

Migrate provider-result specialization and occurrence-local metadata to the typed path first. Keep the old renderer fitting as a fallback. Success means that the current provider structural-elimination, contextual-assignment, implicit-forall, and higher-order-result regressions pass without using the legacy inference path.

17.3 Milestone 2: semantic fingerprinting and shared preparation (M)

- Intern neutral types, substitutions, contexts, and family IDs.
- Compute candidate fingerprints before rendering.
- Deduplicate each engine and combined mode by fingerprint.
- Share provider/certificate/family preparation across engines.
- Add dominance pruning over obligation/loss/cost vectors.

This milestone should reduce Lean verification attempts and make the existing 60/12/24 windows materially more valuable without changing search power.

17.4 Milestone 3: adaptive occurrence-plan tail (L)

- Assign stable IDs to positive forall occurrences.
- Preserve the historical plan prefix exactly.
- Add normalized plan fingerprints and a best-first tail under a distinct compiled-view budget.
- Re-express the twelve-site six/six sentinel as an adaptive success.
- Add larger adversarial cases to demonstrate bounded behavior without adding fixed sextic code.

Structured verifier feedback can be added after a deterministic engine-only heuristic proves the architecture.

17.5 Milestone 4: generalized certificates and family descriptors (L)

- Lift provider assignments into a common certificate table while preserving the existing public constructors as adapters.
- Introduce family descriptors for current intrinsic and query-wide finite/recursive families.
- Move total arity, structural/nominal identity, constructor maps, phantom fidelity, and source rendering handles into descriptors.
- Reimplement `Prod`/`Sum`, tuples, `Either`, `List`, and `Nat` through the common path.

No new user-visible family need be added until the current families pass through the descriptor path. This is an abstraction-validation milestone.

17.6 Milestone 5: source obligations and recursor skeletons (L/XL)

- Generalize residual constraints to obligations.
- Add bounded evidence-table nodes and Lean source completion.
- Serialize non-dependent recursor descriptors for `Nat`, `List`, and simple user inductives.
- Generate one-recursor branch skeletons and solve branches with existing engines.
- Mark omitted recursors as projection loss for negative evidence.

This milestone should produce the first qualitatively new class of synthesized programs rather than only broader higher-rank transport.

17.7 Milestone 6: dependent structures and indexed refinement (XL)

Proceed through D0–D3 from Section 11. Do not begin with arbitrary dependent recursors. The first acceptance criteria should be exact dependent transport, `Sigma`/`Subtype` construction and projection, and one-layer index-refining cases with source equality obligations.

17.8 Milestone 7: semantic provider index and portfolio cooperation (L/XL)

The retrieval index can be developed in parallel after stable family/type fingerprints exist. Progress-aware scheduling and leaf filling should wait for typed candidate graphs and shared subgoal keys.

Table 3: Priority matrix. Impact is on synthesis reach and architectural leverage; risk is implementation/semantic risk.

Direction	Impact	Effort	Risk	Dependency
Typed candidate graph + occurrence IDs	Very high	L	Medium	None
Semantic candidate fingerprinting	High	M	Low	Typed nodes
Adaptive rank-N plan tail	High	L	Medium	Occurrence IDs
General source certificates	Very high	L	Medium	Typed nodes
Family descriptor algebra	Very high	L	Medium	Certificates helpful
Evidence/equality obligations	High	L	Medium	Typed nodes
Recursor skeletons	Very high	L/XL	Medium	Families + obligations
Dependent/indexed refinement	Very high	XL	High	All preceding semantic layers
Persistent semantic provider index	High	L	Low–med.	Stable fingerprints
Cooperative leaf filling	High	XL	Med.-high	Typed DAG + shared cache
Refutation certificates	Medium	L	Medium	Capability model

Recommendation 7: make Milestone 1 the next architectural project and use one current provider-result regression as its vertical slice.

A good first slice is a provider application whose exact first term argument specializes a later rank-N domain, followed by structural elimination of the provider result. Today that path exercises mixed-spine recovery, substitution, fitting, occurrence metadata, and explicit binder weaving. Carrying the same information in typed nodes would demonstrate the new interface against a hard, already well-tested case.

18 Evaluation and regression strategy

18.1 Measure semantic work, not only wall time

Interactive wall time matters, but it is a noisy aggregate of pure search, subprocess scheduling, Lean elaboration, cache state, and machine load. Each benchmark should also report deterministic semantic counters:

- source nodes serialized and opaque atoms introduced;
- exact family descriptors and certificates admitted/rejected;
- unique rank-N plans compiled versus raw plans proposed;
- unique formula and prepared-premise fingerprints;
- Djinn sequents/rule branches and Exference steps/queue prunes;
- shared subgoal cache hits;
- raw candidate derivations, semantic fingerprints, and dominated terms;
- source alternatives generated and verification attempts;

- verification outcomes by failure class;
- obligations created, solved, and rejected; and
- capability losses attached to the final outcome.

A change is an improvement if it reduces expensive semantic work or extends the solved corpus under the same explicit budgets, even when local wall-clock noise hides the gain.

18.2 Benchmark dimensions

Build a matrix rather than a flat list of showcase types.

Quantifier planning. Vary:

- independent positive occurrence count from 1 through at least 20;
- required open/opaque balance, especially central antichains;
- nesting depth and shared ancestor paths;
- duplicated occurrences that compile to the same formula;
- and interaction with query-correlated or closed instantiations.

The current ten-, eleven-, and twelve-site sentinels should remain fixed anchor points. Add cases where many raw plans normalize to few formula views to test semantic budgeting.

Provider certificates. Vary:

- binder width, kinds, and vacuity;
- complete versus partial vectors;
- multiple vectors per provider;
- nested quantified/contextual arguments;
- canonical structural and nominal family heads;
- phantom, mixed faithful/phantom, recursive, and empty datatypes;
- occurrence-local rendering alternatives; and
- conflicting or stale source metadata.

Every positive should have a malformed or semantically unsafe neighbor which fails closed without aborting unrelated evidence.

Candidate identity. Construct queries with many equivalent derivations:

- repeated providers with alpha-equivalent types;
- eta-expanded and contracted forms;
- tuple construction followed by projection;
- source alternatives differing only in explicit binder spelling;

- equivalent synthetic premise paths; and
- Djinn/Exference candidates normalizing to the same graph.

Assert the number and order of semantic fingerprints separately from rendered spellings.

Recursive and dependent stages. For recursor work, vary constructor count, branch arity, recursive fields, independent SCCs, and nested recursor depth. For dependent stages, vary exact transport, dependent pair construction/projection, index-refining cases, definitional versus propositional equality, and deliberately unsupported motives.

Environment retrieval. Use synthetic environments containing controlled distractors as well as real Lean projects. Record recall of a known minimal provider set at each semantic retrieval stage, inventory size, search work, and verified result rank.

18.3 Property-based and metamorphic tests

The current hand-crafted regressions are strong. Typed interfaces enable additional systematic properties.

Alpha invariance.
Renaming source binders or private neutral identities does not change semantic candidate fingerprints or outcomes, modulo source display names.

Occurrence separation.
Duplicating a provider use creates distinct occurrence IDs and permits independent metadata choices without conflating substitutions.

Projection round trip.
Typed candidate → legacy `Expression` → typed checking preserves the root neutral type for the compatibility fragment.

Certificate permutation resistance.
Reordering certificate storage does not change semantic results; reordering the telescope inside a certificate is rejected or changes its fingerprint.

Bound productivity.
Infinite, cyclic, or excessively wide caller-built lists/kinds/graphs are rejected without forcing beyond the advertised bound.

Prefix preservation.
Enabling an adaptive tail, source obligations, or cooperative search leaves every historical candidate prefix byte-for-byte or fingerprint-for-fingerprint unchanged until the old frontier is exhausted.

Capability monotonicity.
Adding a positive-only capability can add candidates or weaken a negative verdict, but cannot create a stronger refutation.

Frontend independence.
Source rendering metadata changes spelling but not neutral typing or search identity unless it is attached to a semantically distinct certificate.

18.4 Fuzzing the checked boundary

The certificate/family/typed-graph parsers are security-like boundaries even in a local tool: they consume untrusted caller-built lazy Haskell values and machine-generated source metadata. Add bounded generators for:

- cyclic and shared graphs;
- duplicate IDs and dangling references;
- malformed kinds and arity conflicts;
- scope escape and binder capture;
- inconsistent field types;
- certificate owners from another session;
- branch refinements escaping their branch;
- and terms whose annotation disagrees with the neutral checker.

The expected result is deterministic rejection with a focused diagnostic, not an exception, nontermination, or contamination of sibling evidence.

18.5 Golden tests without freezing accidents

Keep exact golden transcripts for source-visible behavior, especially candidate ordering and explanatory notes. For internal architectural changes, add a parallel semantic golden format containing fingerprints, obligations, losses, and normalized term graphs. This avoids forcing incidental private names or administrative lambdas when the user-visible output remains equivalent.

19 Risks and mitigations

19.1 The typed graph could become a second source language

Risk. Adding binders, sorts, indices, equality, recursors, and source metadata may grow the neutral IR until it resembles Lean's full expression language.

Mitigation. Keep source terms and indices opaque; add only distinctions required by a generic search rule; make every source-specific operation an obligation or certificate; and require at least two frontend uses, or a clear engine-independent invariant, before moving a fact into the shared core.

19.2 Two representations can drift

Risk. During migration, `Frag`, shared `Type`, typed candidates, and legacy `Expression` may encode overlapping semantics.

Mitigation. Define one-way checked projections and round-trip properties. Do not maintain independent hand-written interpretations in each engine. Mark compatibility adapters explicitly and migrate one vertical slice at a time.

19.3 Adaptive planning could destabilize candidate order

Risk. A best-first tail changes which large-plan candidate appears first and may be sensitive to heuristic details.

Mitigation. Preserve the complete historical prefix; use stable source-order tie-breaking; version the tail policy; expose deterministic plan fingerprints in diagnostics; and treat heuristic order beyond the old frontier as experimental.

19.4 Source feedback could accidentally affect soundness

Risk. Structured Lean failures used for refinement may be mistaken for proof that a plan is impossible.

Mitigation. Feedback changes only positive-search priority. It may never eliminate a completeness-required plan or strengthen negative evidence. Capability and refutation code should not depend on verifier failure classes.

19.5 Obligation search could explode

Risk. Allowing class, equality, coercion, and universe holes may turn one term skeleton into a huge source completion space.

Mitigation. Seal finite evidence tables; cap obligations and source alternatives per candidate; rank obligation-free candidates first; deduplicate by obligation fingerprint; and use one outer command deadline exactly as the current staged searches do.

19.6 Family descriptors may trust too much frontend data

Risk. A malformed descriptor could give an engine an unsound constructor or eliminator rule.

Mitigation. Validate all neutral structural invariants; treat source-only facts as obligations; independently check every positive candidate; and exclude descriptor-dependent refutations until the descriptor class has a checked completeness proof. Lean-derived descriptors can include a compact source fingerprint and be regenerated after environment changes.

19.7 Persistent retrieval can become nondeterministic

Risk. Learned weights or cache history may make results hard to reproduce.

Mitigation. Store transparent counts; version index formats; include index generation in transcripts; provide a lexical/rating-only mode; and make all tie-breaking stable. Cache deletion may affect speed and ranking, never soundness.

20 Directions not recommended as the primary next step

Do not make the next major project merely "raise six to seven."

The shared bound is well-factored, so a seven-binder widening may be a cheap and useful incremental change. It does not address the semantic recovery tax, quantifier-plan lattice, dependent structure, or environment composition. It should be treated as maintenance, not the strategic roadmap.

Do not add sextic, septic, and higher fixed frontiers indefinitely.

Keep the quintic family as a deterministic prefix and validation oracle. Put new coverage in a normalized adaptive tail whose budget is unique compiled views. Otherwise each central antichain creates another bespoke scheduler and documentation burden.

Do not move raw Lean syntax into Djex as semantic authority.

Exact source text is fragile under qualification, pretty-printing options, implicit arguments, binder visibility, universes, and notation. Carry stable IDs and structural metadata; let Leant build Lean syntax at the final edge.

Do not rely on blind generate-and-reject as the main dependent-type strategy.

Lean verification makes blind search safe, not effective. Explicit typed skeletons and obligations should constrain candidate generation before the expensive elaborator call.

Do not merge Djinn and Exference into one monolithic algorithm.

Their independent completeness and ranking semantics are valuable. Share interned artifacts, candidate identity, and typed leaves; cooperate through a portfolio protocol instead of erasing the distinction.

Do not expose stronger non-inhabitation wording merely because the source checker rejected every generated variant.

Rejected candidates are positive-search evidence only. Negative evidence must continue to come from a complete logical projection with an explicit capability fingerprint.

21 Overall assessment

The recent development of Djex and Leant is exceptionally coherent. The repositories are not duplicating work. They form a two-stage research and engineering system:

- Leant supplies difficult, source-realistic Lean cases and an independent semantic oracle;
- Djex converts the reusable part of each case into a checked, frontend-neutral search capability;
- Leant then proves that the abstraction was neither too weak to render nor too permissive to trust.

That method should continue. The architectural unit of progress, however, needs to change. The last several capability loops increasingly preserve semantic provenance through parallel maps and recover types after search. The next abstraction should make provenance, expected types, substitutions, family identity, and residual source obligations first-class in the candidate itself.

With that layer in place, the future directions reinforce one another:

1. typed nodes enable semantic deduplication and shared subgoal caches;
2. stable occurrence IDs enable adaptive quantifier planning and exact source metadata;
3. generalized certificates unify provider, class, family, and equality evidence;
4. family descriptors support canonical higher-kinded identity, recursive recursors, and indexed refinement;
5. obligations let Lean solve source-specific evidence without infecting the neutral engine;
6. semantic retrieval finds the small provider set needed for complex compositions; and
7. a portfolio can let Djinn solve structural leaves while Exference selects environment bridges.

The likely result is not a universal synthesizer for arbitrary Lean types. It is something more useful and technically defensible: a substantially larger, explicitly bounded class of complex higher-order, higher-rank, constrained, recursive, and lightly dependent types for which the system can construct verified terms automatically, explain what semantic resources it used, and say precisely when a negative result is or is not justified.

Final recommendation

Make a typed candidate graph with occurrence identity, substitution provenance, and residual obligations the next shared architectural milestone. Validate it by migrating one of the current hardest provider-result/structural-elimination paths. Then use that foundation for semantic deduplication, an adaptive rank-N plan tail, generalized source certificates, and family descriptors. Those four steps convert the successful Djex–Leant feedback loop from a sequence of careful special cases into a platform for the next generation of synthesis rules.

A Representative commit map

The following list is not exhaustive. It records the commits most directly supporting the architectural inferences in this report.

A.1 Djex

Commit	Theme	Architectural significance
d728719f	Quintic rank-N frontiers	Fixed-degree plan family reaches complete coverage through eleven independent sites under a finite view cap.
227bdc52	Closed local instantiation	Positive-only query-local scheme specialization at closed query subtrees.
077f767f	Query-correlated instantiation	Chooses supplied quantified types only when the complete specialized body occurs in the query.
db25170b	Scalar aggregate elimination	Exposes checked tuple structure from globals and provider evidence to Exference.
e10f7f01	Mixed application spine	One shared constructor for term and visible-type arguments, preserving order and wide-spine strictness.
03aa3820	Contextual exact assignments	Closed contextual rank-N vectors with occurrence-sensitive fidelity marking.
103fdd39	Sibling-field fidelity	Prevents one faithful field from masking a selected argument erased in another field.
c5afc68a	Kinded shared classes	Preserves authoritative class-parameter kinds at the neutral environment boundary.
66955da7	Six-binder widening	One public bound advances hypothesis, loaded-scheme, and provider routes together.
5a550eb4	Structural higher-kinded assignment	Connects bare pair-constructor evidence to the same identity as saturated tuple structure.
f50b5eac	Reviewed head	Merges canonical structural higher-kinded assignment support.

A.2 Leant

Commit	Theme	Architectural significance
e09bded3	Explicit provider heads	Reconstructs explicit forall placeholders for discovered provider applications.

Commit	Theme	Architectural significance
15fb0962	Nested fitting traversal	Reaches provider applications beneath lambdas, tuples, unknown applications, and eliminations.
690277ca	Rank-N fields through elimination	Transfers exact provider result fragments into case/let-bound fields.
892117dc	Result specialization from terms	Infers quantified fragment substitutions from exact term arguments.
77ee65b3	Full-spine specialization	Propagates learned substitutions to later higher-rank arguments.
888e2b40	Exact forall rendering metadata	Retains visibility and `Prop`/`Type`/`Sort` domain tags beside canonical fragments.
fe4b2932	Occurrence-coupled metadata	Ensures the metadata used to fit one provider occurrence is the metadata later rendered there.
2979dbe2	Structured contextual assignments	Carries exact nominal Lean class contexts through private engine classes and source-faithful rendering.
1668c956	Higher-kinded context arguments	Preserves residual kind arity and total nominal-family identity.
d04c1a8f	Six-binder integration	Derives every bridge width from the shared Djex authority and verifies all-engine Lean cases.
0dbe08a7	Structural higher-kinded synthesis	Separates canonical nominal evidence from legacy fragment payloads and maps `Prod`/`Sum` structurally.
0160237e	Reviewed head	Merges structural higher-kinded synthesis and pins Djex at `f50b5eac`.

B Suggested initial API migration

A deliberately incremental API sequence is:

1. Add `OccurrenceId`, `CertificateId`, and `FamilyId` as validated, collision-safe neutral identifiers.
2. Add `TypedExpression` as an annotation wrapper around current `Expression` nodes rather than replacing the syntax immediately.
3. Let engine details optionally carry a `TypedCandidate` and keep `candidateOutput` unchanged.
4. Add a neutral checker and semantic fingerprint for the typed path.
5. Extend Leant's provider maps to be keyed directly by occurrence and certificate IDs.
6. Migrate one application/elimination vertical slice.
7. Once all public engines emit typed nodes for a constructor, make the legacy renderer inference for that constructor diagnostic-only.
8. Generalize residual constraints to obligations under a new API version.
9. Introduce family descriptors and adaptive plans after typed occurrence IDs are stable.

A compatibility envelope might be:

```
data CandidateDetails backend ty source = CandidateDetails
  { backendDetails :: backend
  , typedCandidate :: Maybe (TypedCandidate ty LocalId source)
  , capabilityInfo :: CapabilityInfo source
  }
```

Existing callers ignore `typedCandidate`; `Leant` opts in immediately; new neutral tooling can inspect it without knowing Djinn or Exference internals.

C Seed benchmark types

The following schematic families are suitable for a future benchmark corpus. They should be instantiated in both Haskell-like neutral tests and Lean goldens where the source language permits.

Central rank-N antichains.
Twelve or more independent positive quantified sites where the witness requires a balanced open/opaque choice not present in the fixed quintic prefix.

Correlated higher-kinded assignments.
Providers with two to six binders assigned to a mixture of `Prod`, partially applied `Sum`, user constructors, and closed quantified types, including exact contextual class arguments.

Metadata-sensitive repeated uses.
The same provider used twice with alpha-equivalent canonical types but different explicit/implicit source binder spellings.

Higher-order result specialization.
An early exact term argument determines a later rank-N function domain and the provider returns a product whose polymorphic field is eliminated.

Phantom-fidelity adversaries.
Nested aliases and mixed faithful/phantom sibling fields which would permit an invalid visible specialization if any occurrence marker were lost.

Recursor skeletons.
`Nat` iteration, `List` fold, a binary tree catamorphism, and a two-constructor user family with one recursive field.

Dependent structures.
`Sigma`/`Subtype` projection and reconstruction, followed by one-layer indexed cases whose impossible branch requires an equality obligation.

Environment bridges.
Synthetic libraries where the solution requires exactly one, two, or three providers separated by distractors with the same result head.

Portfolio leaf filling.
A structural Djinn skeleton with one provider leaf; an Exference provider choice with two pure structural argument holes; and a mixed case with a source evidence obligation.

D Primary repository sources

The analysis used the following current repository materials in addition to the commit history linked above:

- Djex README and its linked architecture, API, REPL, and dated implementation reports;
- shared type representation;
- shared generated syntax;
- candidate envelope;
- query and provider-evidence bounds;
- Leant README;
- Leant synthesis proposal and implementation status;
- Lean fragment serializer/parser;
- engine boundary and scheduling; and
- source fitting, rendering, and candidate specialization.

End of report.